

What is RBAC?

Definition

RBAC (Role-Based Access Control) is a security mechanism where permissions are given based on a user's role.

Instead of saying:

```
Sayyami can delete bikes.
```

```
Rahul cannot.
```

```
Akshay cannot.
```

We say:

```
ADMIN can delete bikes.
```

```
USER cannot.
```

Notice the difference.

We don't assign permissions to people.

We assign permissions to **roles**.

Our Bike Management System

Let's design one.

There are only two roles.

```
ADMIN
```

```
USER
```

USER

Can:

Login
View Bikes
Create Bikes
Update Own Bike

Cannot:

Delete Bikes
Delete Users

ADMIN

Can:

Login
View Bikes
Create Bikes
Update Bikes
Delete Bikes
Delete Users
Manage Everything

Authentication

Question:

Who are you?

Example:

Email

Password

Correct?

Yes.

Server says:

Welcome Rahul.

Done.

Authentication is complete.

Authorization

Now another question.

Rahul

Can you delete users?

Server checks.

Rahul is

USER

Permission?

NO

Server says

403 Forbidden

This is Authorization.

Step 1: User Sends a Registration Request

The frontend sends:

```
POST /register
```

Request Body:

```
{
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "password": "rahul123"
}
```

Notice:

There is **no role**.

Why?

Because the client should never decide whether they are an admin.

Step 2: Pydantic Schema

Open `schemas.py`

```
from pydantic import BaseModel, EmailStr

class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str
```

Why is there no **role**?

If you added:

```
role:str
```

then anyone could send:

```
{
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "password": "123",
  "role": "ADMIN"
}
```

Now Rahul becomes an admin.

That is a serious security problem.

So we **do not** expose the role in the registration request.

Step 3: SQLAlchemy Model

Open `models.py`

```
from sqlalchemy import Column, Integer, String
from database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String, unique=True)
    password = Column(String)
    role = Column(String, default="USER")
```

Now every user object has:

```
user.role
```

Step 4: Password Hashing

Suppose you already have:

```
from passlib.context import CryptContext

pwd_context=CryptContext(
    schemes=["bcrypt"],
    deprecated="auto")

def hash_password(password:str):
    return pwd_context.hash(password)
```

his code sets up Passlib to hash passwords securely using the bcrypt algorithm.

```
from passlib.context import CryptContext
```

What is CryptContext?

CryptContext is a class provided by the Passlib library that manages password hashing.

Instead of calling bcrypt directly, you use a CryptContext, which:

hashes passwords, verifies passwords, allows you to change hashing algorithms later without changing the rest of your code.

Think of it as a password manager for your application.

```
pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)
```

This creates a password hashing configuration.

```
schemes=["bcrypt"]
```

This tells Passlib:

"Use the bcrypt algorithm for hashing passwords."

bcrypt is one of the most widely used password hashing algorithms because it is:

Slow (which makes brute-force attacks difficult)

Automatically generates a random salt

Secure and battle-tested

```
deprecated="auto"
```

Imagine today you use bcrypt. After 5 years, a better algorithm comes out, such as Argon2.

You can update your context:

```
pwd_context = CryptContext(
    schemes=["argon2", "bcrypt"],
```

```
    deprecated="auto"  
)
```

Now:

New passwords use Argon2.0ld bcrypt passwords still work.

After a user logs in, Passlib can automatically rehash their password with Argon2.

That's what deprecated="auto" is for.

Never store:

```
rahul123
```

Instead store:

```
$2b$12$A8f7x...
```

Step 5: CRUD Function

Open `crud.py`

This function is responsible for saving the user into PostgreSQL.

```
from argon2 import hash_password  
from sqlalchemy.orm import Session  
from models import User  
from schema import UserCreate  
  
def create_user(db: Session, user: UserCreate):  
  
    # Convert plain password into hashed password  
    hashed_password = hash_password(user.password)  
  
    # Create SQLAlchemy User object  
    new_user = User(  
        name=user.name,  
        email=user.email,  
        password=hashed_password,  
        role="USER"  
    )
```

```
# Save to database
db.add(new_user)

# Commit transaction
db.commit()

# Refresh object to get generated fields like id
db.refresh(new_user)

return new_user
```

Step 6: Registration Route

Open `routes.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from database import get_db
from schemas import UserCreate, UserResponse
from crud import create_user

router = APIRouter()

@router.post("/register", response_model=UserResponse)
def register(user: UserCreate, db: Session = Depends(get_db)):
    return create_user(db, user)
```

-  Create a reusable `require_role()` dependency.
-  Understand why it returns another function (**closures**).
-  Use `Depends(require_role("ADMIN"))`.
-  Protect APIs.

Problem

Suppose we write


```
@router.delete("/users/{id}")
def delete_user(id: int, current_user=Depends(get_current_user)
):

    if current_user.role != "ADMIN":
        raise HTTPException(
            status_code=403,
            detail="Access Denied"
        )

    return {"message": "Deleted"}
```

This works.

But imagine your application has

```
DELETE /users
DELETE /bikes
DELETE /orders
DELETE /products
DELETE /employees
DELETE /payments
DELETE /customers
```

Every endpoint would contain

```
if current_user.role!="ADMIN":
```

Again.

Again.

Again.

Again.

Hundreds of times.

This violates one of the most important software engineering principles:

Don't Repeat Yourself (DRY).

Instead of copying the same code everywhere, we create it **once** and reuse it.

Solution

Create one function.

Then use it everywhere.

This function will be called

```
require_role()
```

Where Should It Be?

Create a new file.

```
backend/  
  
dependencies.py
```

Step 1 — Create require_role()

Open

```
dependencies.py
```

Write

```
from fastapi import Depends  
from fastapi import HTTPException  
from auth import get_current_user
```

Now create

```
def require_role(role:str):  
    pass
```

Question:

Why are we passing

```
role
```

?

Because sometimes we need

```
ADMIN
```

Sometimes

```
USER
```

Sometimes

```
MANAGER
```

Sometimes

```
EDITOR
```

The required role changes depending on the API.

Step 2 — Create the Inner Function

Now write

```
def require_role(role:str):  
    def role_checker():  
        pass  
    return role_checker
```

Most beginners stop here and ask:

Why are we creating a function inside another function?

Great question.

Let's understand.

Normal Function

Example

```
def greet():  
    print("Hello")
```

Call

```
greet()
```

Output

```
Hello
```

Simple.

Function Returning Function

Now

```
def outer():  
    def inner():  
        print("Hello")  
        return inner
```

Question:

What does

```
x=outer()
```

store?

Answer

```
The inner function itself.
```

Not the result.

The function.

Now

```
x()
```

Output

```
Hello
```

Let's visualize it.

```
Call outer()  
↓  
Create inner()  
↓
```

Return inner()

↓

Store in x

↓

Later

↓

x()

↓

Runs inner()

This concept is called a **closure** in Python.

What is a Closure?

A closure is:

An inner function that remembers variables from its outer function, even after the outer function has finished executing.

Example:

```
def outer():  
    name = "Sayyami"  
    def inner():  
        print(name)  
    return inner
```

Now

```
x=outer()
```

Question:

`outer()` has already finished.

Why can `inner()` still print `name` ?

Because it **remembers** the variable from the outer function.

```
x()
```

Output

```
Sayyami
```

That "remembering" behavior is what makes closures so useful.

How Does This Apply to RBAC?

Now look at

```
def require_role(role):  
    def role_checker():  
        print(role)  
    return role_checker
```

Call

```
admin=require_role("ADMIN")
```

What happens?

```
role = "ADMIN"

↓

Create role_checker()

↓

Return role_checker
```

Even after `require_role()` finishes, `role_checker()` still remembers:

```
role = ADMIN
```

Now

```
admin()
```

Output

```
ADMIN
```

Try another one.

```
manager=require_role("MANAGER")
```

Now

```
manager()
```

Output

```
MANAGER
```


The same function works for different roles because each returned function remembers the value it was created with.

Step 3 — Use Depends()

Now let's implement the real dependency.

```
from fastapi import Depends, HTTPException

from auth import get_current_user

def require_role(role: str):

    def role_checker(

        current_user=Depends(get_current_user)

    ):

        if current_user.role != role:

            raise HTTPException(

                status_code=403,

                detail="Access Denied"

            )

        return current_user

    return role_checker
```

This is the complete reusable dependency.

Let's Understand Every Line

Outer Function

```
def require_role(role: str):
```

Input:

```
require_role("ADMIN")
```

or

```
require_role("MANAGER")
```

It **does not** check the role yet.

It simply creates a customized dependency.

Inner Function

```
def role_checker(current_user=Depends(get_current_user)):  
    :
```

FastAPI sees

```
Depends(get_current_user)
```

and automatically runs:

```
current_user=get_current_user()
```

Suppose JWT belongs to Rahul.

Then

```
current_user
```

contains

```
User(id=2,name="Rahul",role="USER")
```

Check Permission

```
if current_user.role!=role:
```

Suppose

```
role="ADMIN"
```

Current user

```
current_user.role="USER"
```

Python compares

```
"USER"!="ADMIN"
```

Result

```
True
```

Raise

```
HTTPException(status_code=403,detail="Access Denied")
```

Return User

If roles match

```
return current_user
```

Now the endpoint can still use the logged-in user.

Step 4 — Protect an API

```
@router.delete("/users/{id}")
```

```
def delete_user(
```

```
    id: int,

    current_user=Depends(
        require_role("ADMIN")
    ):

```

```
    return {
        "message": "User Deleted"
    }

```

FastAPI performs:

```
require_role("ADMIN")
|
▼
Returns role_checker()
|
▼
Runs get_current_user()
|
▼
Reads JWT
|
▼
Loads User from PostgreSQL
|
▼
```

Checks `current_user.role`

